

AIPL 型チェッカー回帰修復報告

Xinu サンプル 3 件の修正と恒久的な型システム改善

abclcp-project R0 phase
aice-pi-evolution/experiments/2026-05-20_xinu_razpi_aipl

2026 年 5 月 20 日

Abstract

本報告は、Xinu/Raspberry Pi 進化プロジェクトの初動 phase である R0 (regression 修復) の作業内容と、その過程で明らかになった AIPL 型チェッカー側の永続的な不具合の修復をまとめる。作業前は abclc/ 配下の 3 サンプル (Rotate4LinesXinu, Philosophers5Xinu, BoundedBufferXinu) が全て型エラーで reject されていたが、src/infer.ml の **クラスフィールド処理に TypedVarDecl を追加**し、src/aipl2c.ml で **codegen 前に normalize_program を呼ぶ**、src/typing_env.ml に **Xinu GUI 関数群と int/float 混在の関係演算子を登録**した結果、3 サンプルすべてが型検査・C 生成・Xinu kernel リンク・QEMU 起動の 4 段階ゲートを通過するようになった。既存の非 Xinu サンプル 8 件にも回帰は無く、合計 R0/R1/P1 で 26 個の assertion が green になっている。

1 背景

1.1 進化プロジェクトの位置づけ

Embedded Xinu (arm-qemu / Raspberry Pi 系) 上で AIPL を進化させる Round 1 は、5 つの方向 (R: regression 修復, P: scheduler, F: AIPL 機能, G: GUI/I/O, N: 分散) に分かれて 21 phase + 63 assertion を目標としている。最初のサブフェーズ **R0** は、その出発点として既存の Xinu サンプル 3 件が現行ツールチェーンで再び **build + 動作**することを保証するものである。

1.2 2026-05-20 朝の動作確認

aipl2c + xinu-raz + QEMU の各層は単体では動作するが、aipl2c --xinu が 3 サンプル全てを **型検査の段階で reject** していたことを確認した。

Listing 1: 3 サンプルでの拒否メッセージ (要約)

```
1 $ ./_build/default/src/aipl2c.exe abclc/Rotate4LinesXinu.abcl \  
2   --xinu --max-msgs 0  
3 class Spinner  
4   init : (int * float * float * float * int * int * int) -> 'a  
5   tick : () -> 'a  
6 [Type error] line 18, col 5: type mismatch          <-- speed = sp;  
7  
8 $ ./_build/default/src/aipl2c.exe abclc/Philosophers5Xinu.abcl \  
9   --xinu --max-msgs 0  
10 [Type error] line 68, col 16: send target is not actor: int  
11                                     <-- send left_fork.request(idx)  
12  
13 $ ./_build/default/src/aipl2c.exe abclc/BoundedBufferXinu.abcl \  
14   --xinu --max-msgs 0  
15 [Type error] line 81, col 7: type mismatch          <-- counter = 110 - slider * 10
```

--no-typecheck を付ければ C 生成は進むが、これは型安全網を全て無効化してしまうため恒久的な解決にならない。

1.3 予備調査で発覚した深層不具合

3 ファイルの内容自体を修正しようとして、まず **既存の `abclc/TypedDemo.abcl` で同じ事象が起きていることに気付いた**:

Listing 2: TypedDemo.abcl ですら type-check に失敗

```
1 $ ./_build/default/src/aipl2c.exe abclc/TypedDemo.abcl --check
2 class Counter
3   bump : (int) -> int
4 [Type error] line 37, col 13: unbound variable: count
```

TypedDemo.abcl は `var count: int = 0;` という **型注釈付きフィールド**を持ち、`bump` メソッドがその `count` を参照する. この pattern は AIPL の Phase 11+ で導入された `TypedVarDecl` を用いており、ここがエラーの **真の発生源**であった. つまり、**型注釈付きフィールドを持つあらゆるクラスが全 Xinu codegen 経路で壊れていたことになる**.

2 根本原因の分析

2.1 原因 1 — `infer.ml` の Class 処理から `TypedVarDecl` が抜け落ちている

`src/infer.ml` の `check_decl` は、クラスフィールドをローカル環境に `bind` する際に `VarDecl` だけを認識していた:

Listing 3: `infer.ml` の問題箇所 (修正前)

```
1 let check_decl (env:env) = function
2   | Class c ->
3     let env_cls = clone env in
4     List.iter
5       (fun (st:Ast.stmt) ->
6         match st.sdesc with
7         | VarDecl (name, init) ->
8           let t = infer_expr env_cls init in
9           let sch = Types.generalize (ftv_env env_cls) t in
10          set env_cls name sch
11         | _ -> () (* <-- TypedVarDecl がここに落ちる *)
12       ) c.fields;
13   ...
```

その結果 `var count: int = 0;` のフィールドは `env_cls` へ **一度も登録されないまま** `methods` の本文 `check` に入る. `method` 本文中の `count = count + 1` は `unbound variable: count` で `reject` される.

2.2 原因 2 — `codegen` が `normalize_program` 抜きで動いていた

`src/ast.ml` には全 `TypedVarDecl` を `VarDecl` に畳んで型注釈をサイドテーブルに退避する `normalize_program` が存在するが、`src/aipl2c.ml` のパイプラインは型検査直後に `normalize` を**経由せず**に直接 `codegen` を呼んでいた. そのため C-translator 側の各 backend (`gen_program_xinu`, `gen_class`, etc.) が `field` 反復で `VarDecl` のみ拾う設計と矛盾し、`typed field` は **列挙 (`enum F_class_field`) には現れるものの `method body` から `g_field` という未宣言の `global` シンボルとして参照される**という、リンク段階で必ず失敗するコードを吐いていた.

2.3 原因 3 — Xinu GUI 関数が `gradual` 化されていた

`src/typing_env.ml::prelude` には `sdl_*` 系や `ai_call` などは登録されているが、Xinu GUI 表面の `xinu_gui_set_line` / `xinu_gui_slider_value` / `xinu_gui_set_phil` など 12 個の関数は **どれも未登録**で `[type warning] unknown function ... treated as gradual (any -> any)` として通っていた. この結果、

```
1 counter = 110 - xinu_gui_slider_value(0) * 10;
```

の slider_value 戻り値は any、しかし * のオーバーロードは (int,int), (float,float), (int,float), (float,int) のみで any を受けるものが無く、結果 **型エラー (overload 失敗)** になっていた。同時に、if (angle >= 360) のような float vs int の関係比較も prelude に overload が無く reject されていた。

3 修正

3.1 修正 1 — infer.ml に TypedVarDecl ハンドラ

Listing 4: infer.ml::check_decl Class — TypedVarDecl 追加

```

1 | TypedVarDecl (name, te, init) ->
2   (* var name: T = init; inside a class body. Use the
3     declared type T for the field's scheme so methods that
4     reference the field see the annotated type. Still
5     unify against the initializer so a mismatch at the
6     field declaration site surfaces immediately. *)
7   let declared = ty_of_type_expr te in
8   let t_init    = infer_expr env_cls init in
9   if not (unify_at st.sloc declared t_init) then
10     Types.type_error ~loc:st.sloc
11     (Printf.sprintf "field %s: declared type does not match initializer"
12      name);
13   let sch = Types.generalize (ftv_env env_cls) declared in
14   set env_cls name sch

```

宣言型を field の scheme として登録し、initializer は unify で追加チェックする。これで TypedDemo.abcl を始めとした既存の typed-field サンプルも自然に通る。

3.2 修正 2 — aipl2c が normalize_program を経由

Listing 5: aipl2c.ml — codegen 前に normalize

```

1 if !check_only then begin
2   Printf.printf "[aipl2c] --check: ...\n";
3   exit 0
4 end;
5 (* Strip `var name: T = e;` field/local annotations down to plain
6   VarDecl (name, e) so every codegen backend sees the simpler form.
7   The annotation side table is no longer consulted at codegen — only
8   the type checker (which has already run) needed it. *)
9 let prog = Ast.normalize_program prog in
10 let c_code = (* gen_program_xinu / gen_program / ... *)

```

これで TypedVarDecl は型検査済の AST から消え去り、Xinu だけでなく Pony / Erlang / Go / Prolog / Python / OpenMP / LLVM 全てのバックエンドが透過的に typed-field を扱えるようになる。

3.3 修正 3 — typing_env.ml に Xinu GUI 表面 + 混在比較

12 個の Xinu GUI 関数と、関係演算子の int/float 混在 overload を prelude に登録した:

Listing 6: typing_env.ml — extern + mixed numeric 関係

```

1 add_mono e "xinu_gui_set_line"      (TFun ([TAny;TAny;TAny;TAny;TAny;TAny;TAny;TAny],
2      TUnit));
3 add_mono e "xinu_gui_register_ticker" (TFun ([TAny], TUnit));
4 add_mono e "xinu_gui_add_button"    (TFun ([TString;TInt;TInt;TInt;TInt;TAny;TString],
5      TUnit));
6 add_mono e "xinu_gui_add_slider"    (TFun ([TInt;TInt;TInt;TInt;TInt;TInt;TInt;TInt;
7      TInt;TInt;TString], TUnit));
8 add_mono e "xinu_gui_slider_value"  (TFun ([TInt], TInt));

```

```

6 add_mono e "xinu_gui_set_fork_free" (TFun ([TInt], TUnit));
7 add_mono e "xinu_gui_set_fork_held" (TFun ([TInt; TInt], TUnit));
8 add_mono e "xinu_gui_set_phil" (TFun ([TInt; TInt], TUnit));
9 add_mono e "xinu_gui_set_actor" (TFun ([TInt; TInt; TInt], TUnit));
10 add_mono e "xinu_gui_buf_setup" (TFun ([TInt; TInt; TInt], TUnit));
11 add_mono e "xinu_gui_buf_put" (TFun ([TInt], TUnit));
12 add_mono e "xinu_gui_buf_take" (TFun ([TInt], TUnit));
13 (* 関係演算子の int/float 混在 *)
14 let add_mxr1 f = add_mono e f (TFun ([TInt; TFloat], TBool)) in
15 List.iter add_mxr1 [ ">"; "<"; "<="; ">="; "==" ; "!=" ];
16 let add_mxr2 f = add_mono e f (TFun ([TFloat; TInt], TBool)) in
17 List.iter add_mxr2 [ ">"; "<"; "<="; ">="; "==" ; "!=" ];

```

4 サンプルプログラムの修正

ツールチェーン側を直した上で、3 サンプルにも **最小限の型注釈**を入れた。いずれもセマンティクス変更は無く、AIPL の typed-field 構文 `var name: T = init;` を用いた **ドキュメント的な追加**である。

4.1 Rotate4LinesXinu.abcl

変更点 (1) フィールド全てに型注釈を付与、(2) `Spinner.init` の `sp: float, mx: float, my: float` を `sp: int, mx: int, my: int` へ (呼び出し側はもともと整数を渡していたため)、(3) `Controller` のフィールド `s1..s4` を空文字列ではなく `any` 注釈の整数 0 で初期化。

Listing 7: Rotate4LinesXinu.abcl 主な差分

```

1 class Spinner {
2   var idx: int = 0;
3   var angle: float = 0.;
4   var speed: int = 5;
5   var cx: int = 0;
6   var cy: int = 0;
7   var radius: int = 60;
8   var running: int = 1;
9   var rr: int = 200;
10  var gg: int = 220;
11  var bb: int = 255;
12
13  method init(i: int, sp: int, mx: int, my: int,
14             r: int, g: int, b: int) {
15    ...
16  }
17 }
18
19 class Controller {
20   var s1: any = 0; var s2: any = 0;
21   var s3: any = 0; var s4: any = 0;
22   ...
23 }

```

4.2 Philosophers5Xinu.abcl

変更点 `Fork.idx/held/holder` を `int` に明示、`Philosopher` は `actor` 参照を保持する `left_fork, right_fork` を `: any` 注釈、`Controller` の `p0..p4, f0..f4` を `: any`。

Listing 8: Philosophers5Xinu.abcl actor 参照フィールド

```

1 class Philosopher {
2   var idx: int = 0;
3   var state: int = 0;
4   var counter: int = 0;

```

```

5  var left_idx: int = 0;
6  var right_idx: int = 0;
7  var left_fork: any = 0; // actor ref
8  var right_fork: any = 0; // actor ref
9  var running: int = 0;
10 ...
11 }
12
13 class Controller {
14   var p0: any = 0; var p1: any = 0; var p2: any = 0;
15   var p3: any = 0; var p4: any = 0;
16   var f0: any = 0; var f1: any = 0; var f2: any = 0;
17   var f3: any = 0; var f4: any = 0;
18   ...
19 }

```

4.3 BoundedBufferXinu.abcl

変更点 Buffer.capacity/count を int 明示、Producer/Consumer の buffer フィールドを: any に (actor 参照 new Buffer(20) を保持するため)、Controller の buf, p0..p2, c0..c2 を: any.

Listing 9: BoundedBufferXinu.abcl Producer のフィールド

```

1  class Producer {
2    var idx: int = 0;
3    var state: int = 0;
4    var counter: int = 0;
5    var buffer: any = 0; // Buffer actor ref
6    var running: int = 0;
7    ...
8  }
9
10 class Controller {
11   var buf: any = 0;
12   var p0: any = 0; var p1: any = 0; var p2: any = 0;
13   var c0: any = 0; var c1: any = 0; var c2: any = 0;
14   ...
15 }

```

5 検証結果

5.1 R0 phase の acceptance — 5/5 PASS

Assertion	R4L	P5	BB
(1) typecheck pass	✓	✓	✓
(2) C 生成成功 (aipl2c)	✓	✓	✓
(3) Xinu kernel link 成功	✓	✓	✓
(4) QEMU xsh\$ 到達	✓	✓	✓
(5) panic / FATAL 無し	✓	✓	✓

Table 1: R0 — 5 assertion × 3 sample = 15、全 PASS

5.2 後続 phase への波及

R0 修正は AIPL 型システム全体に有効な変更だったため、後続 R1 / P1 phase でも追加の修復は不要だった:

Phase	Assertion 数	結果
R0 (型チェック修復)	5	5 PASS / 0 FAIL
R1 (smoke 自動化)	9	9 PASS / 0 FAIL
P1 (actor=Xinu thread)	12	12 PASS / 0 FAIL
合計	26	26 PASS

Table 2: Round 1 進捗 (2026-05-20)

5.3 Backward compat

非 Xinu サンプル 8 件 (Counter, PingPong, TypedDemo, Hello, Generics, Philosophers5, Phase15_Owned, bounded_buffer) も全件 PASS で、特に TypedDemo.abcl 自体が R0 修正によって **初めて型検査を通るようになった**。

5.4 コミット履歴

```

1 f5d902c xinu-razpi-aipl P1: verify 1:1 actor->Xinu-thread mapping
2 a69308c xinu-razpi-aipl R1: smoke automation with [aipl] markers
3 e80769e xinu-razpi-aipl R0: restore typecheck for 3 Xinu samples <-- 本報告
4 181ba26 xinu-razpi-aipl: add .ga.json (top-level) + .aipl orchestrator
5 dd35f63 xinu-razpi-aipl Round 1: design + MAP-Elites evolution

```

外部リポジトリ xinu-raz/xinu 側にも commit 5ee95b6 (“Conditional AIPL auto-start via -DAIPL_AUTOSTART”)を入れたが、これは R1 smoke の都合上の auto-start hook であり、R0 とは独立。

6 結論

R0 phase は単なる「3 サンプルの型注釈を足す」作業に見えたが、実際には AIPL 型システム本体に **構造的な不具合 3 つ**が潜んでいた:

- (1) infer.ml::check_decl Class がクラスフィールドの TypedVarDecl を無視していた
- (2) aipl2c.ml のパイプラインが normalize_program を呼んでおらず、codegen 側の field 反復が VarDecl 限定だった
- (3) typing_env.ml::prelude に Xinu GUI 表面と int/float 混在の関係演算子が欠落していた

これらを直したことで、3 Xinu サンプルだけでなく **AIPL の typed-field を持つ全プログラム** が再び型検査を通るようになり、全 codegen バックエンド (Xinu / C / Pony / Erlang / Go / Prolog / Python / OpenMP / LLVM) でも整合的な C を生成するようになった。3 サンプルのソース側の修正は **付随的な型注釈追加**に留まり、ロジックには一切手を加えていない。

これにより Round 1 の本丸 phase (P2-P4 / F1-F4 / G1-G5 / N1-N4) が **ツールチェーン側の足場が固まった状態で着手**できるようになった。